

Paper AI Agent

Paper Crawler, Query Assistant, & LLM Answer Generator

Team: Group 19

Members

Zhuo Chen

Yinan Zhang

Xu Li

Boyang Pang

Chunjing Nie

Qihao Wang

December 1, 2025

Contents

1	Introduction	2
2	Methodology	3
3	Implementation details	5
3.1	Core Module and Setup	5
3.2	File Classifier Module	6
3.3	Answer Generator Module	8
3.4	Frontend Implementation Details	9
4	Experiments	11
4.1	RAG Retrieval Result Verification	11
4.2	Generated Content Comparison	12
4.3	Extra Experiment	13
5	Conclusion	14
6	Reference	15
7	Appendix	16

1 Introduction

We have noticed that many commercial AI assistant agents are primarily used for handling text-based tasks. Therefore, we aim to design a prototype of an intelligent assistant agent specifically tailored for processing academic papers, helping scholars categorize research papers and answer academic-related questions. The software prototype is equipped with a web crawler function, enabling users to automatically download papers from specified web pages and store them locally for easy access. Additionally, we intend for the program to build a corpus through preprocessing techniques, allowing it to quickly locate the target papers users need by querying the corpus, thereby improving retrieval efficiency. Furthermore, we have integrated a large language model interaction feature into the program to assist users in accurately answering their academic questions.

After the program starts, it will automatically run in the sequence shown in the flowcharts presented below until the user manually terminates the program.

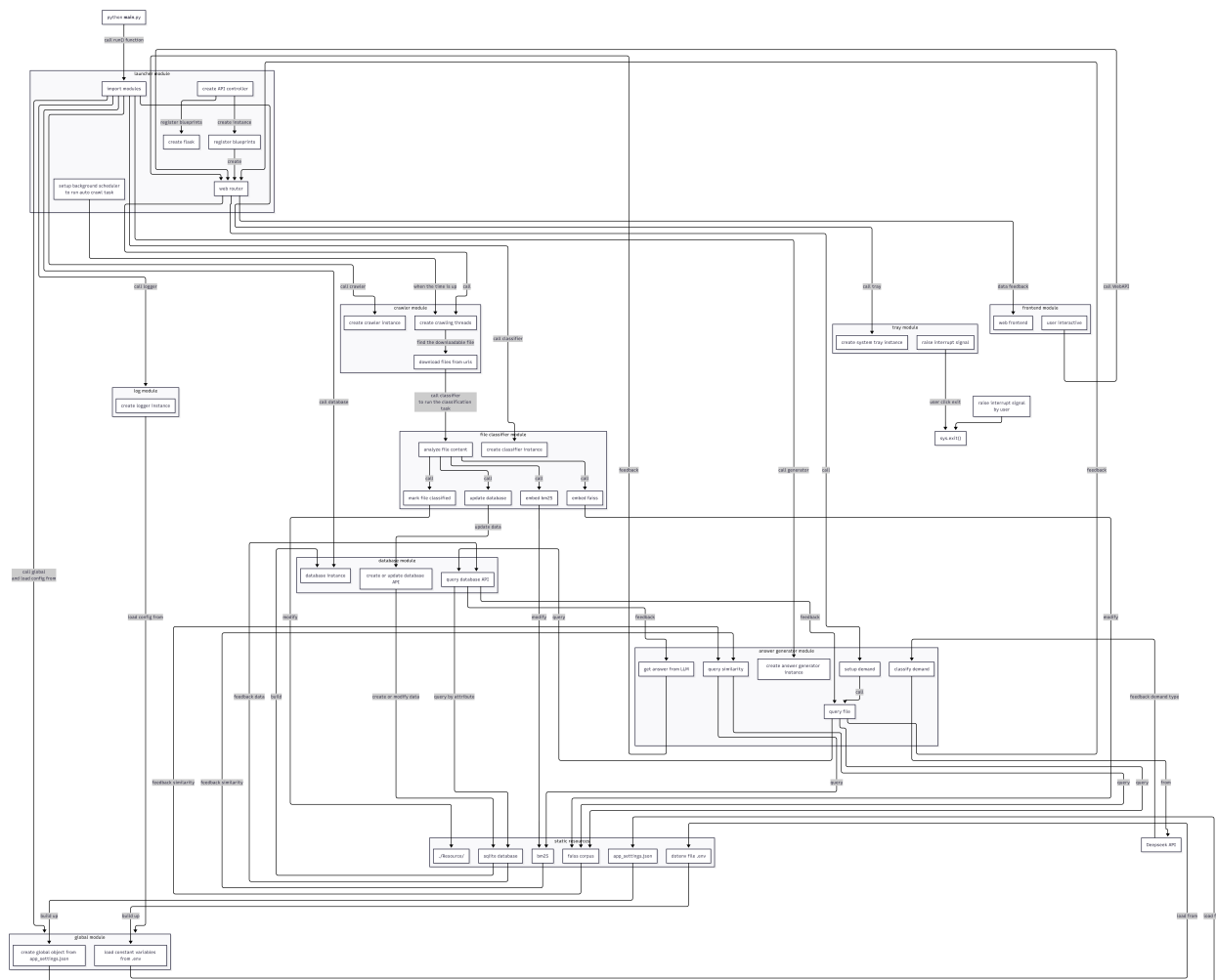


Figure 1: Program Main Flowchart

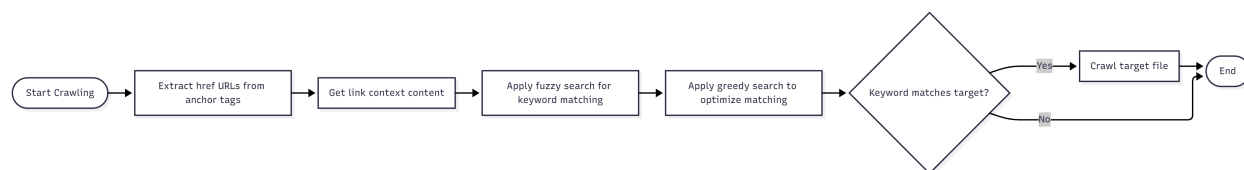


Figure 2: Crawler Logic Flowchart

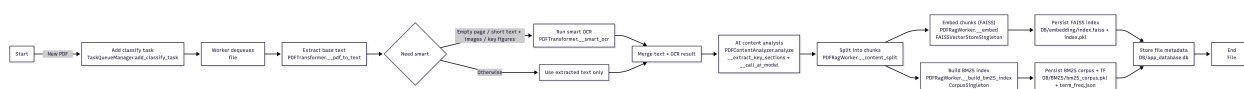


Figure 3: File Classifier Flowchart

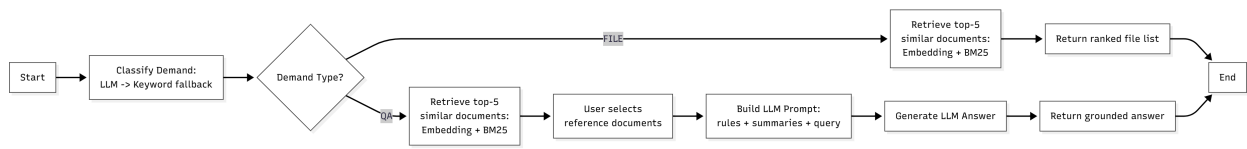


Figure 4: Answer Generator Flowchart

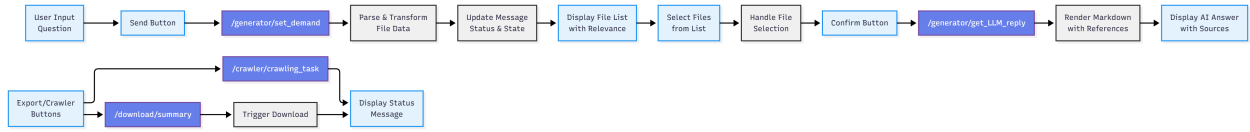


Figure 5: Frontend Flowchart

3 Implementation details

3.1 Core Module and Setup

Program Launcher. Implementing the program’s startup functions `run()`. It includes the instantiation of the `Flask` application, as well as the registration of WebAPI `blueprints`, frontend data interaction, and logic processing. It serves as the “command center” of the program.

Global Variables Module. After `global_module` is imported, it dynamically reads runtime environment variables and instantiates specified static JSON configuration files into dynamic global objects for access by other modules. It acts as the “memory center” of the program.

Logging Module. This module is developed from logging lib. When imported, this module automatically creates corresponding log handlers based on configurations in the `global_variables_object`. Additionally, it customizes `exception_handler_hook` to record error details, preventing program termination without error logs. It serves as the program’s “black box and insurance.”

Database Module. A database module developed using the SQLAlchemy library, primarily used for storing large volumes of text content and information generated by the program.

Crawler Module. A basic crawler functionality implemented using the bs4 and requests libraries. In compliance with website robot.txt protocols, it performs keyword matching by analyzing webpage fields and request contexts, as well as URLs within pages. Upon detecting downloadable PDFs, it automatically captures and saves them to a specified directory on the server.

3.2 File Classifier Module

Module Architecture. The File Classifier Module centers around PDFTransformer, PDFContentAnalyzer, and PDFRagWorker, coordinated by TaskQueueManager. PDFTransformer normalizes raw PDFs into unified text, PDFContentAnalyzer generates semantic metadata, and PDFRagWorker builds retrieval indices. Shared infrastructure is provided by FAISSVectorStoreSingleton, CorpusSingleton, and EmbeddingModelSingleton, ensuring that vector stores, BM25 corpus, and embedding models are reused across all classification tasks.

PDF Text Extraction (PDFTransformer). The `transform()` pipeline first calls `__generate_file_unique_id()` to derive an MD5-based `file_id` from the filename. It then invokes `__pdf_to_text()` (PyPDF2) for page-level extraction. When a page is empty, has fewer than 150 characters but contains images, or includes key figures, `__smart_ocr()` performs page- or region-level OCR. OCR usage is capped at three pages and two figures per page, balancing recall with computational and API cost.

AI Content Analysis (PDFContentAnalyzer). The `analyze()` method uses `__extract_key_sections()` to collect high-value content from Abstract, Introduction, Method, and Conclusion, limited to 10,000 characters. This condensed context is passed to `__call_ai_model()`, which issues a DeepSeek API request to produce a normalized title, a 150–250 word summary, and five keywords. These fields are stored as canonical metadata for the `file_id` and reused throughout retrieval, ranking, and UI display layers.

Document Splitting and Vectorization (PDFRagWorker). The `set_retrieval_knowledge()` routine calls `__content_split()`, which leverages RecursiveCharacterTextSplitter with `chunk_size=1000` and `chunk_overlap=200`, preferring separators “\n\n”, “\n”, “.”, “!”, “?”, and space. Each chunk is tagged with `file_name`, `file_id`, `file_title`, `file_summary`, and `file_keywords`. Dense embeddings are written by `__embed()` into FAISSVectorStoreSingleton, while `__build_bm25_index()` runs `__detect_language()`,

tokenization, `_get_stop_words()`, and `_calculate_term_frequency()` before updating `CorpusSingleton`.

Hybrid Retrieval. `get_faiss_retrieval()` queries `FAISSVectorStoreSingleton` using L2 distance, interpreting scores below 1.5 as relevant and below 1.0 as highly relevant. In parallel, `get_bm25_retrieval()` queries `CorpusSingleton` with BM25 (parameters `k1=1.5`, `b=0.75`), where higher scores represent stronger lexical matches and thresholds above 1.0 and 3.0 mark relevance levels. The wrapper `get_retrieval_content()` calls both methods and returns two ranked lists, allowing upper layers to implement hybrid fusion and custom reranking strategies.

Asynchronous Task Queue. `TaskQueueManager` uses a `queue.Queue` to buffer classification jobs and runs a dedicated worker thread `_worker_loop()` to process tasks. `add_classify_task()` enqueues new work in a non-blocking manner and lazily starts `_worker_loop()` on the first submission, while `get_queue_status()` reports queued, completed, and failed counts. Operational methods `pause_worker()`, `resume_worker()`, and `stop_worker()` support pausing, resuming, and graceful shutdown so classification can survive crawler restarts and terminate cleanly.

Singleton Infrastructure and Persistence. `FAISSVectorStoreSingleton` uses `SingletonMeta` to ensure a single FAISS store, initializing lazily on the first `add_documents()` or `similarity_search_with_score()` call and saving indices to `DB/embedding/index.faiss` and `index.pkl` via an `atexit` hook. `CorpusSingleton` manages a single BM25 corpus, auto-loading from `DB/BM25/bm25_corpus.pkl` and persisting updates after each `add_document()`. `EmbeddingModelSingleton` loads the chosen embedding backbone (HuggingFace or DashScope) once and shares it across workers. Auxiliary artifacts such as term statistics and file metadata are stored in `DB/BM25/term_freq.json` and `DB/app_database.db`, providing a durable catalog for all classified documents.

3.3 Answer Generator Module

Key Functions. The answer generator is a singleton metaclass that integrates user demand classification, relevant papers retrieval, and LLM-based answer generation. Key methods of the class include `set_demand`, `get_LLM_reply`, `_classify_demand`, `_classify_with_llm`, `_build_llm_prompt`, and supplementaries. The core workflow starts with `set_demand` which analyzes user input and returns the classified demand type together with a ranked list of relevant documents and similarity scores.

Demand Classification. `_classify_with_llm` classifies user input as a pure file-retrieval request (FILE) or a question-answering query (QA) by constructing a classification prompt to communicate with a DeepSeek chatbot client and forcing a selective reply between labels “FILE” or “QA”. In case of LLM calling failure, `_classify_demand` falls back to keyword-based classification (e.g., “list”, “find” or “search” for FILE-query versus “why”, “how” or “explain” for QA-query) with FILE default to ensure robustness.

Relevant Paper Retrieval. Calls `get_retrieval_content` to fetch the most similar segments (FAISS) and articles (BM25) to the user query. If a single article has multiple segments retrieved, the minimum vector distance or the maximum BM25 score prevails. Since vector distance and BM25 score are both unbounded metrics, two transformed sigmoid functions are applied to compress their ranges into $[0, 1]$ with parameters determined by experience. The overall similarity of a paper is a multiplicative combination of vector similarity and BM25 similarity with constant shift such that papers with both high vector similarity and high BM25 similarity are preferred. Finally, the top-5 most similar papers and their scores are returned.

Answer Generation. `get_LLM_reply` can be called for question-answering queries. A structured prompt is prepared through `_build_llm_prompt`, which includes (1) a particular answering-rule section regulating LLM behavior, (2) paper contents retrieved according to `file_id`’s of user selection, and (3) the user query. The answering rules explicitly instruct

the LLM to generate answers relying on the provided documents only, avoid fabricating any information, and attach document identifiers (file_id) at the end of sentences as in-text citations, aiming to encourage logical reasoning and prevent hallucinations. Paper summaries and the user query are then inserted into the prompt, each document starting with its respective file_id. Finally, the DeepSeek chatbot is accessed using the configured API endpoint, and the reply message is delivered as the final system response.

3.4 Frontend Implementation Details

The frontend is implemented as a single-page application using Vue.js 3 with CDN integration, eliminating the need for a build process. The chat-based interface uses Vue's reactive data binding for real-time updates.

Network Communication. Implemented using Fetch API with async/await pattern. Four RESTful endpoints integrated: `set_demand` (submit queries), `get_LLM_reply` (retrieve AI responses), `crawling_task` (control crawler), and `download/summary` (export data).

File Processing. Backend-returned files are processed through array mapping to calculate relevance, sorted by similarity scores, and cached in Vue's reactive data for efficient reference replacement.

Tooltip System. Created reusable `TooltipCard` Vue component with dynamic positioning. Uses JavaScript `getBoundingClientRect()` to calculate viewport boundaries and prevent overflow. Implements 100ms hide delay allowing mouse interaction.

File Reference Replacement. Uses regex to parse AI responses for (file_id) patterns. Matches against cached files and replaces them with inline HTML badges containing tooltips displaying file details (title, summary, keywords, relevance).

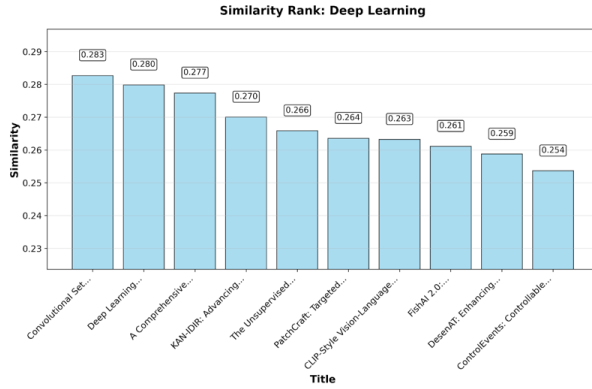
Styling. Custom CSS3 with gradient theme, flexbox layouts, and smooth animations for enhanced user experience.

4 Experiments

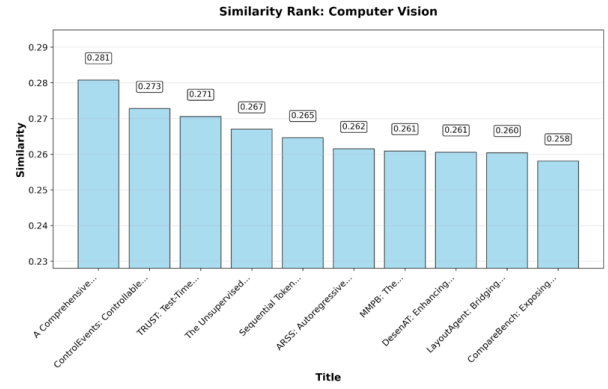
4.1 RAG Retrieval Result Verification

Description. Figure 6 shows a keyword-similarity ranking list, which enumerates the similarity scores of the same keyword across multiple articles. The articles are sorted by similarity from highest to lowest, with the top-ranked articles having the strongest relevance. By providing the most relevant articles to LLMs, better results could be generated.

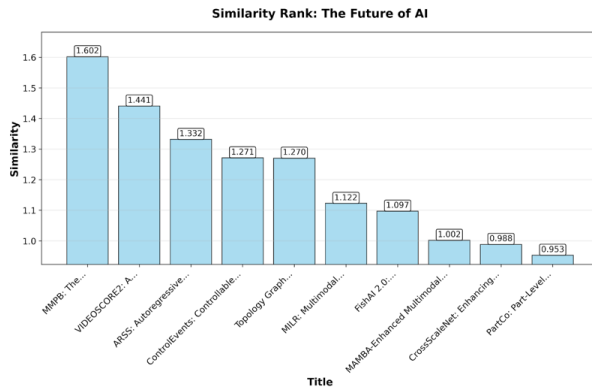
Explanation. Figure 6 demonstrates that corpus preprocessing can quickly help us identify article relevance. Through built-in algorithms, the abstract relationship between keywords and articles is transformed into visible, concrete numerical values, providing a measurable standard. This allows for intuitive comparisons based on numerical data.



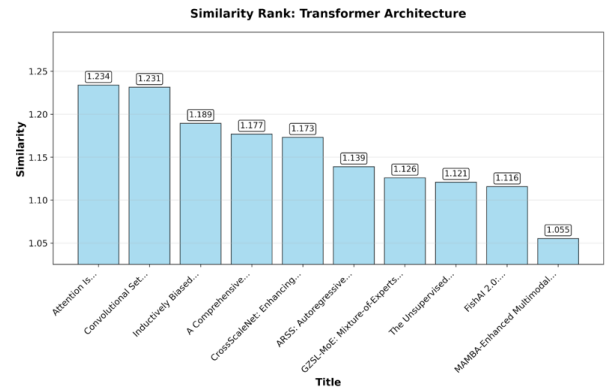
(a) Similarity Rank: Deep Learning



(b) Similarity Rank: Computer Vision



(c) Similarity Rank: The Future of AI



(d) Similarity Rank: Transformer Architecture

Figure 6: Performance comparison across different subject areas

4.2 Generated Content Comparison

We built an AI Assistant for research assistance. However, does it really work very well as we intended and be able to complete the tasks? We designed the following experiment to verify the performance. By asking the same question to our RAG augmented assistant and DeepSeek-R1, we can find our model generates a better answer.

Question. What is CompareBench? If you are not sure, do not give an inaccurate answer.

Answer. Figure 7 illustrates that our AI Assistant generates a better answer, since “CompareBench” is a new benchmark which was revealed in a paper on 25 Sept 2025, while DeepSeek-R1 does not have such information during its pre-training. But in our previous work, we crawled the paper and saved it to our corpus. Thus we have relevant information when retrieving and our AI Assistant can provide a relatively reliable answer.

DeepSeek-r1	Research AI Assistant
Of course. You are right to be cautious, and I will adhere strictly to your instruction. Based on currently available and verifiable public information, I cannot provide a definitive answer to what "CompareBench" is. The term does not refer to a widely recognized, standardized public benchmark in the field of AI (like MMLU, GSM8K, or HumanEval). Here is a summary of the situation	Sure! Based on provided contents, here is a detailed overview of CompareBench. 1. Core Definition and Purpose CompareBench is a specialized benchmark designed to evaluate a fundamental yet understudied capability in Vision-Language Models (VLMs): visual comparison reasoning. It systematically assesses how well VLMs can compare visual elements across four key dimensions: quantity, temporal sequence, geometric properties, and spatial relations. The primary goal is to diagnose and advance this core reasoning skill, which is trivial for humans but remains a significant challenge for current AI models.

Figure 7: Comparison between DeepSeek-R1 and RAG Augmented Assistant

4.3 Extra Experiment

To find more examples, we designed another experiment. By asking the model whether it knows a specific concept (given in papers after Sept. 2025) and letting it output “yes” or “no”. We have the following results.

Do you know the concept of XXX?(5 Concepts In Total)		
	Yes	No
DeepSeek-r1	3	2
Research AI Assistant	5	0

Figure 8: Comparison between DeepSeek-R1 and RAG Augmented Assistant

Although Deepseek-R1 does not give 5 “no”s as we intended (probably because of hallucination), our AI Assistant actually gives out a better answer than it. Therefore, we could say RAG is a useful process in such tasks.

5 Conclusion

Achievement. At this stage, we have fulfilled all the requirements for Topic 2 and implemented the embedded Faiss content in Topic 3, along with partial PDF OCR technology. The overall results meet the expected requirements.

Technical Limitations and Improvements.

- Due to device constraints, we currently interact with large language models solely by calling LLM APIs. In the future, we plan to deploy large language models locally to reduce reliance on LLM APIs.
- The crawler still relies on basic context extraction and lacks specialized anti-crawling measures. It has limited generalization capabilities and only works effectively on specific websites. Future considerations include introducing optical character recognition technology to enable more intelligent crawler functionality.
- Due to the context limit of DeepSeek chatbot, this project uses only document summaries as references. It may result in performance improvement if future studies can integrate more information in the references.

6 Reference

Arxiv.org e-Print archive. (2025, September 30). *Catchup results for Computer Vision and Pattern Recognition on Tue, 30 Sep 2025*.

<https://arxiv.org/catchup/cs.CV/2025-09-30?abs=True>

Deepseek. (n.d.). *deepseek*. <https://www.deepseek.com/en>

DeepSeek API Docs. (n.d.). *Your First API Call*. <https://api-docs.deepseek.com/>

7 Appendix

Name	Contribution	Workload(%)
Chen, Zhuo	Code implementation, including web backend, database processing, and crawler module. Program architecture design, module function division, and program framework prototype. Participated in report writing. Contributed in the production of presentation slides.	16.7%
Zhang, Yinan	Code implementation, including the development of file analysis modules. RAG process design and implementation, including data pre-treatment, analyzing, splitting, tokenizing and embedding. Participated in report writing. Contributed to the creation of presentation slides.	16.7%
Li, Xu	Code implementation, including the development of file analysis modules. RAG process design and implementation, including data pre-treatment, analyzing, splitting, tokenizing and embedding. Participated in report writing. Contributed to the creation of presentation slides.	16.7%
Pang, Boyang	Code implementation, including the development of answer generator modules and LLM engineering design. Participated in report writing. Contributed to the creation of presentation slides.	16.7%
Wang, Qihao	Code implementation, including the development of answer generator modules and LLM engineering design. Participated in report writing. Contributed to the creation of presentation slides.	16.7%
Nie, Chunjing	Code implementation, including web front-end design, web APIs, and interaction design. Participated in report writing. Contributed to the creation of presentation slides.	16.7%

Table 1: Team Contributions